

Working with Real Data & Summary Statistics

GOVT 10: Quantitative Political Analysis

This chapter puts R to work on real political data. We load and manipulate datasets with the core tidyverse verbs, handle missing data honestly, and then summarize what we have: measures of central tendency, measures of spread, and the grouped summaries that let us compare across parties, regions, or years.

1 Working with Real Political Data

Last week you learned the basics of R: how to create variables, build vectors, and inspect a data frame. Those skills were necessary, but they were also a bit abstract. This week we work with a real political dataset, and we develop the tools that turn raw data into something we can actually reason about.

Most political datasets share a common structure. Each row represents an observation (a member of Congress, a survey respondent, a precinct, a country-year), and each column represents a variable measured on that observation (their party, their vote, their age, their turnout rate). When we say “the data,” what we usually mean is this rectangular arrangement of rows and columns. The first job of any analyst is to understand what a row represents and what each column measures.

1.1 A Real Dataset: State Presidential Voting Patterns

Throughout this chapter we use a state-level dataset of recent U.S. presidential elections. The file `state_voting_patterns.csv` records, for each of the 50 states, the Republican and Democratic two-party vote shares across two consecutive presidential elections, along with the state’s broad geographic region and which party won. With only a few columns and one row per state-election, the dataset is small enough to read directly but rich enough to ask real comparative questions: do Western states cluster differently from Southern ones? Which states swung the most between elections?

```
states <- read_csv("../data/state_voting_patterns.csv")
```

1.2 Looking at the Data First

Before we manipulate anything, we look. Two functions help us see what we have: `names()` lists the column names, and `glimpse()` shows us the structure column by column.

```
names(states)
```

```
[1] "state"          "year"          "region"
[4] "republican_vote_share" "democratic_vote_share" "winner"
```

What this code does: The `names()` function returns a character vector of every column name in the data frame. Before you ask any substantive question, you need to know what variables you actually have. A typo in a column name is the single most common source of errors in tidyverse code.

```
glimpse(states)
```

```
Rows: 40
Columns: 6
$ state      <chr> "Alabama", "Alabama", "Alaska", "Alaska", "Arizona", "Ariz~
$ year       <dbl> 2016, 2020, 2016, 2020, 2016, 2020, 2016, 2020, 2016, 2020~
$ region     <chr> "West", "West", "West", "West", "West", "West", "West", "W~
$ republican_vote_share <dbl> 48.31329, 38.88607, 46.02116, 41.39943, 45.56660, 38.93880~
$ democratic_vote_share <dbl> 51.68671, 61.11393, 53.97884, 58.60057, 54.43340, 61.06120~
$ winner     <chr> "Democrat", "Democrat", "Democrat", "Democrat", "Democrat"~
```

We learn several things immediately. We can see how many rows the data has (each is a state-year) and how many columns. We can see the data type of each column: `<chr>` means text, `<dbl>` means a number with decimals, `<int>` means whole numbers. And we get a preview of the first few values, which often catches encoding problems before they become real problems.

1.3 Filtering Rows with `filter()`

Most analytic questions concern a subset of the data. “How Democratic did Western states vote in 2020?” requires us to keep only the Western state-years from 2020. The `filter()` function keeps the rows that meet a condition we specify.

```
democratic_wins <- states %>%
  filter(winner == "Democrat")

nrow(democratic_wins)
```

```
[1] 31
```

What this code does: The pipe operator `%>%` passes `states` into `filter()`. The condition `winner == "Democrat"` keeps only rows where the winning party label matches “Democrat” exactly. Note the double equals sign: a single `=` is for assignment, while `==` is the test for equality. The `nrow()` function then counts how many rows survived the filter.

We can stack multiple conditions inside a single `filter()` call. Commas between conditions mean “AND”, so every condition must be true for a row to be kept.

```
competitive_2020 <- states %>%
  filter(year == 2020,
         republican_vote_share >= 45,
         republican_vote_share <= 55)

nrow(competitive_2020)
```

```
[1] 11
```

What this code does: We keep only rows that are simultaneously from the 2020 election and where the Republican vote share fell between 45% and 55%, a common rough definition of a competitive state. We could also have used `%in%` for set membership, for example `region %in% c("South", "West")`, which is much cleaner than writing `region == "South" | region == "West"`.

1.4 Selecting Columns with `select()`

A real political dataset can have hundreds of columns, most of which we do not need for any single question. The `select()` function keeps only the columns we name. Think of `filter()` as choosing rows and `select()` as choosing columns.

```
key_vars <- states %>%
  select(state, year, winner, democratic_vote_share)
head(key_vars, 4)
```

```
# A tibble: 4 x 4
  state    year winner democratic_vote_share
<chr>   <dbl> <chr>             <dbl>
1 Alabama 2016 Democrat         51.7
2 Alabama 2020 Democrat         61.1
3 Alaska  2016 Democrat         54.0
4 Alaska  2020 Democrat         58.6
```

What this code does: We take `states` and keep only four columns, in the order we listed them. `select()` also reorders columns as a side effect, which is useful when you want the most important variables to appear first.

1.5 Sorting Rows with `arrange()`

The `arrange()` function reorders rows by the values in one or more columns. By default it sorts from smallest to largest; wrap a column in `desc()` to sort the other way.

```
states %>%
  filter(year == 2020) %>%
  arrange(democratic_vote_share) %>%
  select(state, region, democratic_vote_share) %>%
  head(3)
```

```
# A tibble: 3 x 3
  state  region democratic_vote_share
<chr>   <chr>             <dbl>
1 Georgia South         46.2
2 Iowa    Midwest         48.0
3 Florida South         48.6
```

What this code does: This is our first multi-step pipeline. We restrict to the 2020 election, sort from the lowest Democratic vote share to the highest, keep three readable columns, and show the top three rows. Read the pipe operator as the word "then": take the data, then filter, then arrange, then select, then show the head.

To see the most Democratic-leaning states instead, we use `desc()`.

```
states %>%
  filter(year == 2020) %>%
  arrange(desc(democratic_vote_share)) %>%
  select(state, region, democratic_vote_share) %>%
  head(3)
```

```
# A tibble: 3 x 3
  state      region democratic_vote_share
  <chr>      <chr>             <dbl>
1 Connecticut Northeast         61.4
2 Alabama    West              61.1
3 Arizona    West              61.1
```

1.6 Handling Missing Data

In real political data, missing values are unavoidable. A senator did not vote on a bill. A respondent skipped a question. A historical record was lost. R represents missing values with a special symbol: NA (Not Available). And R takes NA seriously, perhaps more seriously than you might expect at first.

```
example_data <- c(10, 5, NA, 8)
mean(example_data)
```

```
[1] NA
```

The mean is NA. This is not a bug. R is telling you that it does not know the true mean of a vector with a missing value, because the missing value could be anything. To force R to ignore the missing value, you pass `na.rm = TRUE`.

```
mean(example_data, na.rm = TRUE)
```

```
[1] 7.666667
```

What this code does: The argument `na.rm = TRUE` stands for "NA remove: TRUE." It tells `mean()` to drop missing values before computing. Most aggregating functions (`median()`, `sd()`, `sum()`, `min()`, `max()`) accept the same argument. **Important:** this does not change your original data. It only changes the calculation.

1.7 Detecting Missing Values with `is.na()`

Sometimes we want to identify which values are missing, count them, or remove them outright. You might be tempted to write `values == NA`, but this does not work.

```
NA == NA
```

```
[1] NA
```

The result is NA, not TRUE. The logic is subtle but consistent: if a value is unknown, R cannot say whether it equals anything, even itself. The function `is.na()` is the correct tool: it returns TRUE if a value is missing and FALSE otherwise.

```
is.na(example_data)
```

```
[1] FALSE FALSE TRUE FALSE
```

What this code does: For each element of the vector, `is.na()` returns TRUE if the element is missing and FALSE if it is not. We can then use these logical values inside `filter()` to keep or drop missing rows.

The exclamation point `!` means “not”, so `!is.na(x)` reads as “x is not missing.” This pattern appears constantly in real analyses.

```
states %>%
  summarise(
    total = n(),
    missing = sum(is.na(democratic_vote_share)),
    complete = sum(!is.na(democratic_vote_share))
  )
```

```
# A tibble: 1 x 3
  total missing complete
<int>   <int>   <int>
1     40      0      40
```

What this code does: We summarize three numbers about the Democratic vote share variable. `n()` counts total rows. `sum(is.na(...))` counts how many are missing (TRUE counts as 1, FALSE as 0). `sum(!is.na(...))` counts how many are present. Even when a dataset turns out to be fully complete (as this one is), running the check is good practice: a mean computed from 90% of cases is one thing, a mean from 10% is something else entirely.

A Common Mistake: Silently Dropping Data

When you use `na.rm = TRUE` or `filter(!is.na(...))` without first counting how much data is missing, you have made a methodological decision without examining its consequences. If 5% of cases are missing, dropping them is usually fine. If 40% are missing, dropping them may bias your results in ways that are hard to anticipate. Always report missingness alongside any statistic you compute.

1.8 Correlation Versus Causation

When we look at political data, we naturally notice that some variables move together. States with higher turnout tend to have higher education levels. Countries with more press freedom tend to have lower corruption. These co-movements can be quantified with a number called the correlation coefficient, written r . It ranges from -1 (perfect negative relationship) to +1 (perfect positive relationship), with 0 meaning no linear relationship.

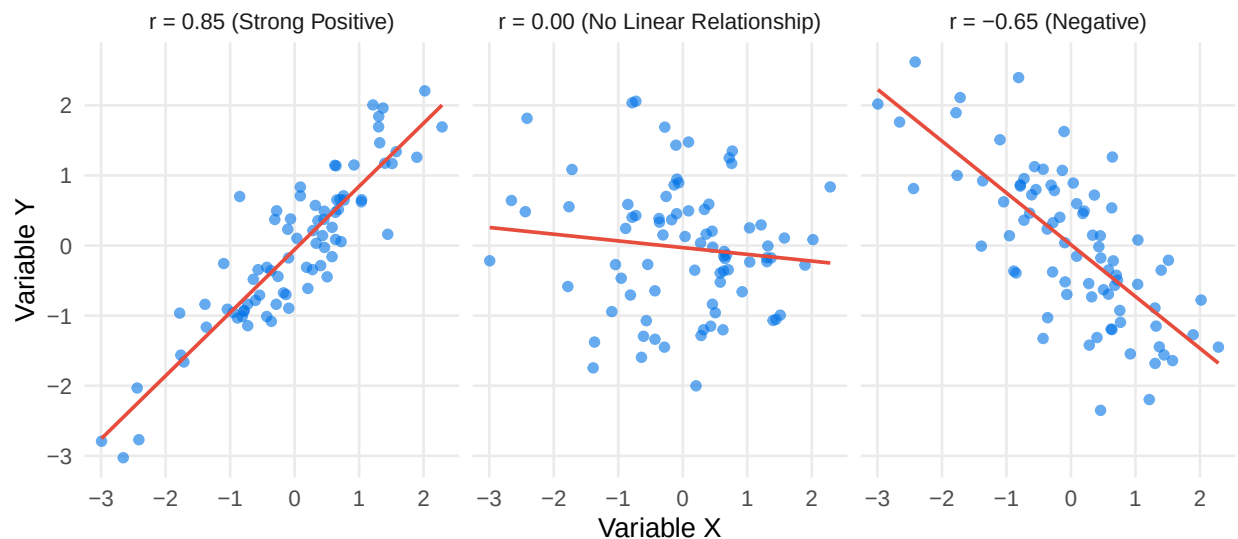


Figure 1: Figure 2.1: Three illustrative correlations. The pattern of points captures the strength and direction of the linear relationship.

A correlation tells us about a pattern. It does not, on its own, tell us anything about cause. This is one of the most important ideas in this course, and it is also one of the easiest to forget.

Consider a famous example. Across countries, per-capita chocolate consumption is positively correlated with the number of Nobel Prize winners. The correlation is real. Does chocolate cause Nobel Prizes? Almost certainly not. National wealth, research investment, and educational infrastructure plausibly cause both. Or consider a political example: states with more police officers per capita tend to have higher crime rates. Does hiring police cause crime? No, the causal arrow runs the other way: high-crime states hire more police.

Critical Misconception: Correlation Implies Causation

When you see a claim that X is "linked to" or "associated with" Y, ask yourself three questions. Could X cause Y? Could Y cause X? Could some third variable Z cause both? Only carefully designed research, particularly randomized experiments, can distinguish among these explanations. We will return to this distinction throughout the course, especially when we turn to causality in Week 4.

2 Summary Statistics

We have learned how to wrangle data: filter rows, select columns, arrange, and handle missing values. But once a dataset is clean, we still need to describe it. A vector of 535 ideology scores or 1,500 approval ratings is too much for any human to absorb at a glance. Summary statistics compress that vector into a few numbers that capture its most important features: where the center

is, how spread out the values are, and what an unusual value would look like.

2.1 Measures of Central Tendency

The first question we usually ask about a distribution is: what is a typical value? There are three standard answers, and they often disagree.

The **mean** is the arithmetic average: add every value and divide by how many you have.

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n} = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

The **median** is the middle value when the data are sorted. Half of observations lie below the median, half above. With an even number of observations, the median is the average of the two middle values.

The **mode** is the value that appears most frequently. For continuous variables like ideology scores, the mode is most useful after binning the data; for categorical variables like party affiliation, the mode is simply the most common category.

```
pres_approval <- read_csv("../data/presidential_approval.csv")
```

```
pres_approval %>%  
  summarise(  
    mean_approval = mean(approval_rating, na.rm = TRUE),  
    median_approval = median(approval_rating, na.rm = TRUE)  
  )
```

```
# A tibble: 1 x 2  
  mean_approval median_approval  
    <dbl>          <dbl>  
1      50.4          48
```

What this code does: We compute two summaries of monthly presidential approval ratings pooled across every president from 1980 to the present. The mean and median tell related but distinct stories. The mean is the balance point. The median is the 50th percentile. When they diverge, the distribution is skewed.

2.2 When Mean and Median Disagree

For a roughly symmetric distribution, the mean and median are close, and either is a fine summary. For a skewed distribution, they diverge, and the choice matters.

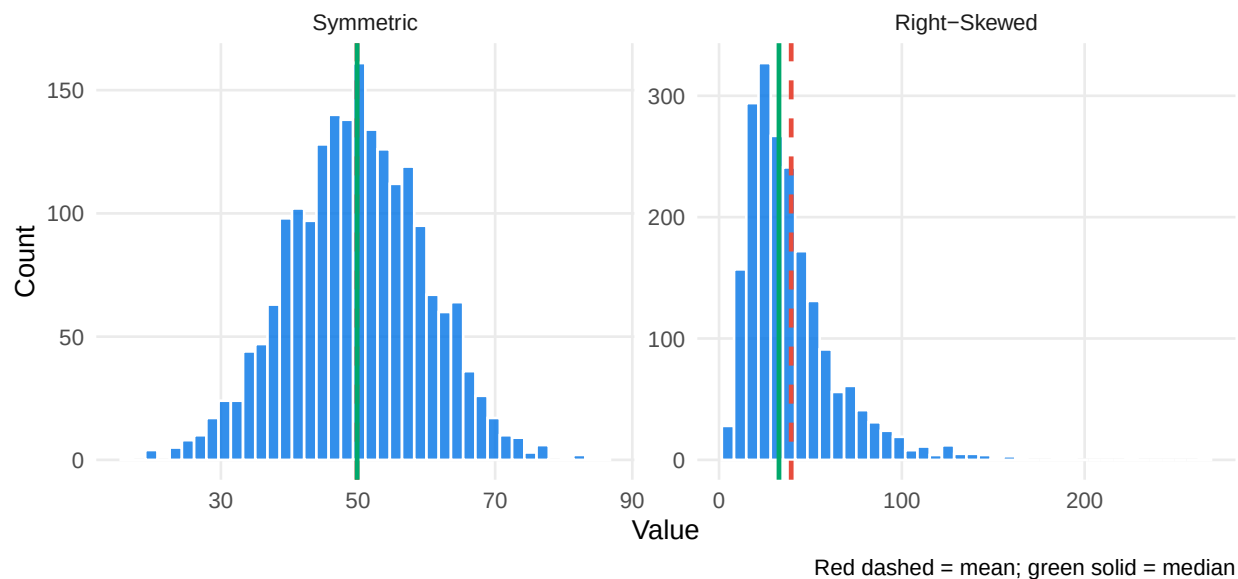


Figure 2: Figure 2.2: A symmetric distribution and a right-skewed distribution. In the skewed case, a long right tail pulls the mean above the median.

The mean is sensitive to extreme values because every observation enters the calculation in proportion to its magnitude. One billionaire pulls the mean wealth of a town far above where the typical resident lives. The median is resistant: changing the largest value from \$1 million to \$1 billion does not move the median at all, because the middle position has not changed.

In political science, many variables are right-skewed: campaign spending, district population, household wealth, and media attention all have long right tails dominated by a few exceptional cases. For these, the median is usually the more honest summary of what is typical. The mean is still useful when the distribution is roughly symmetric, when we plan to do further mathematical analysis, or when the total (sum) of the variable matters substantively, since the mean is just the total divided by n .

Use the Mean When...	Use the Median When...
Distribution is roughly symmetric	Distribution is skewed
No extreme outliers present	Outliers may distort the picture
The total or sum matters	"Typical" value is the goal
Preparing for further inference	Communicating to a general audience

2.3 Measures of Spread

Two distributions can have identical means and look completely different. Consider two midterm distributions:

- Class A: 48, 49, 50, 51, 52 (mean = 50)
- Class B: 10, 30, 50, 70, 90 (mean = 50)

Same center, very different stories. To describe a distribution honestly we also need to say how spread out it is.

The simplest measure is the **range**: the difference between the maximum and the minimum.

```
pres_approval %>%
  summarise(
    min_a = min(approval_rating, na.rm = TRUE),
    max_a = max(approval_rating, na.rm = TRUE),
    range_a = max_a - min_a
  )
```

```
# A tibble: 1 x 3
  min_a max_a range_a
<dbl> <dbl>   <dbl>
1     23    87      64
```

The range is easy to interpret but fragile: it depends entirely on the two most extreme values and ignores everything in between. A single unusual respondent can dramatically change it.

2.4 Variance and Standard Deviation

To use all the data, we measure how far each observation is from the mean, and we average those distances. There is a wrinkle: if we just averaged the raw deviations, positives and negatives would cancel exactly to zero. The fix is to square the deviations before averaging. The result is the **variance**:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \quad (2)$$

The variance is in squared units, which is hard to interpret. (What does a “squared approval percentage” mean?) Taking the square root returns us to the original units; this is the **standard deviation**, s .

```
pres_approval %>%
  summarise(
    mean_a = mean(approval_rating, na.rm = TRUE),
    sd_a = sd(approval_rating, na.rm = TRUE),
    n_obs = n()
  )
```

```
# A tibble: 1 x 3
  mean_a sd_a n_obs
<dbl> <dbl> <int>
1   50.4  11.5   528
```

What this code does: The `sd()` function computes the sample standard deviation. The `n()` function inside `summarise()` counts how many rows went into the calculation. Always report `n`: a standard deviation from several hundred monthly observations means something very different than one from 10 observations.

The standard deviation has a clean interpretation: it is roughly the typical distance of an observation

from the mean. If presidential approval averages 52 across this period with a standard deviation of 11, then a typical month produced an approval rating somewhere around 41 to 63. For approximately bell-shaped distributions, about 68% of observations fall within one standard deviation of the mean, and about 95% within two.

2.5 Grouped Summaries with `group_by()` and `summarise()`

Most interesting political questions are comparative. We rarely care about the mean Democratic vote share across the country as a whole; we care how Western states compare to Southern ones, or how the same region shifted across consecutive elections. The combination of `group_by()` and `summarise()` makes these comparisons easy.

`summarise()` collapses a data frame into a single row of summary statistics. When we add `group_by()` in front, `summarise()` produces one row per group instead.

```
states %>%
  group_by(region) %>%
  summarise(
    n_states = n(),
    mean_dem_share = mean(democratic_vote_share, na.rm = TRUE),
    median_dem_share = median(democratic_vote_share, na.rm = TRUE),
    sd_dem_share = sd(democratic_vote_share, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 4 x 5
  region      n_states mean_dem_share median_dem_share sd_dem_share
<chr>      <int>      <dbl>          <dbl>          <dbl>
1 Midwest         6         51.5           51.0           2.77
2 Northeast        4         59.7           60.8           2.69
3 South           4         46.7           47.4           2.41
4 West           26         55.5           55.3           4.56
```

What this code does: We split the data into groups (one per geographic region), and within each group we compute the count, mean, median, and standard deviation of the Democratic vote share. The argument `.groups = "drop"` removes the grouping after the summary, which keeps later operations predictable. The output lets us compare regions directly: how different are the mean Democratic shares, and how much variation is there within each region?

We can group by multiple variables. The result is one row per combination of group values.

```
states %>%
  group_by(region, year) %>%
  summarise(
    n_states = n(),
    mean_dem_share = mean(democratic_vote_share, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 8 x 4
  region      year n_states mean_dem_share
<chr>      <dbl>    <int>      <dbl>
1 Midwest    2016         3         52.7
```

2	Midwest	2020	3	50.3
3	Northeast	2016	2	58.7
4	Northeast	2020	2	60.8
5	South	2016	2	46.0
6	South	2020	2	47.4
7	West	2016	13	54.7
8	West	2020	13	56.4

The pattern is the same one you will use again and again: filter to the cases you care about, group by the variables that define your comparison, then summarise with whatever statistics answer your question.

2.6 Visualizing Distributions

Numbers alone do not always reveal the shape of a distribution. A histogram or a density plot does. The `ggplot2` package, part of `tidyverse`, lets us layer visual elements onto a coordinate system. Three geoms are essential for this chapter.

`geom_histogram()` divides a numeric variable into bins and shows how many observations fall into each.

```
ggplot(pres_approval, aes(x = approval_rating)) +
  geom_histogram(bins = 25, fill = "#0073E6", color = "white") +
  labs(x = "Approval rating (0-100)", y = "Number of months") +
  theme_minimal()
```

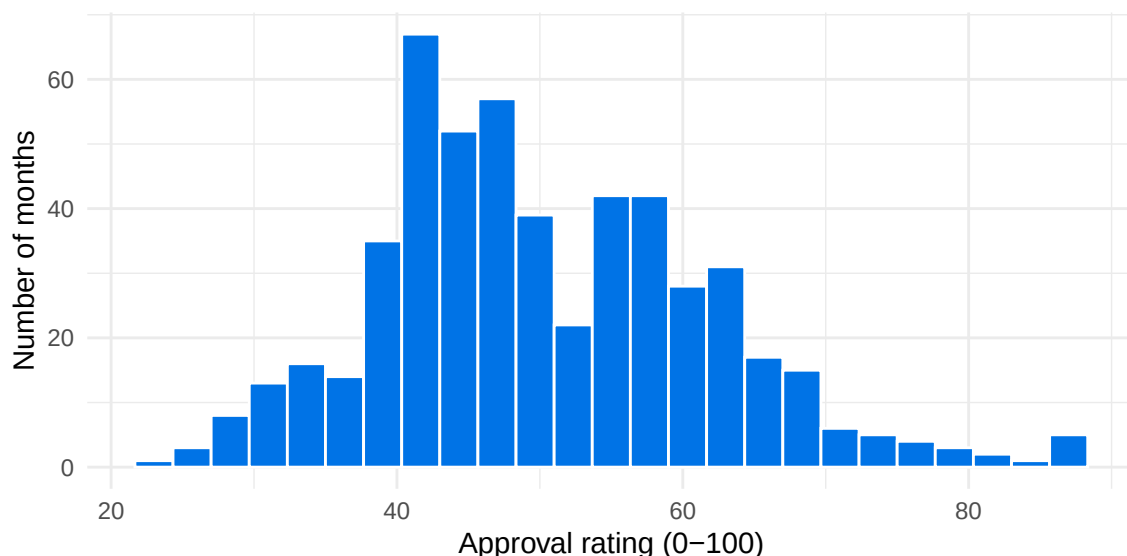


Figure 3: Figure 2.3: Histogram of monthly presidential approval ratings, 1980-present.

What this code does: `ggplot()` sets up the plot and the `aes()` (aesthetic mapping) tells it that the x axis is the approval variable. `geom_histogram()` adds the bars. The `bins` argument controls how many bars to draw; experiment with this number, because it can dramatically change the apparent shape of a distribution.

`geom_density()` draws a smooth curve approximating the same shape. Densities are easier to compare across groups because they do not depend on bin width.

```
mean_a <- mean(pres_approval$approval_rating, na.rm = TRUE)
median_a <- median(pres_approval$approval_rating, na.rm = TRUE)

ggplot(pres_approval, aes(x = approval_rating)) +
  geom_density(fill = "#0073E6", alpha = 0.6) +
  geom_vline(xintercept = mean_a, color = "#E74C3C",
             linetype = "dashed", linewidth = 0.9) +
  geom_vline(xintercept = median_a, color = "#00A86B",
             linewidth = 0.9) +
  labs(x = "Approval rating", y = "Density") +
  theme_minimal()
```

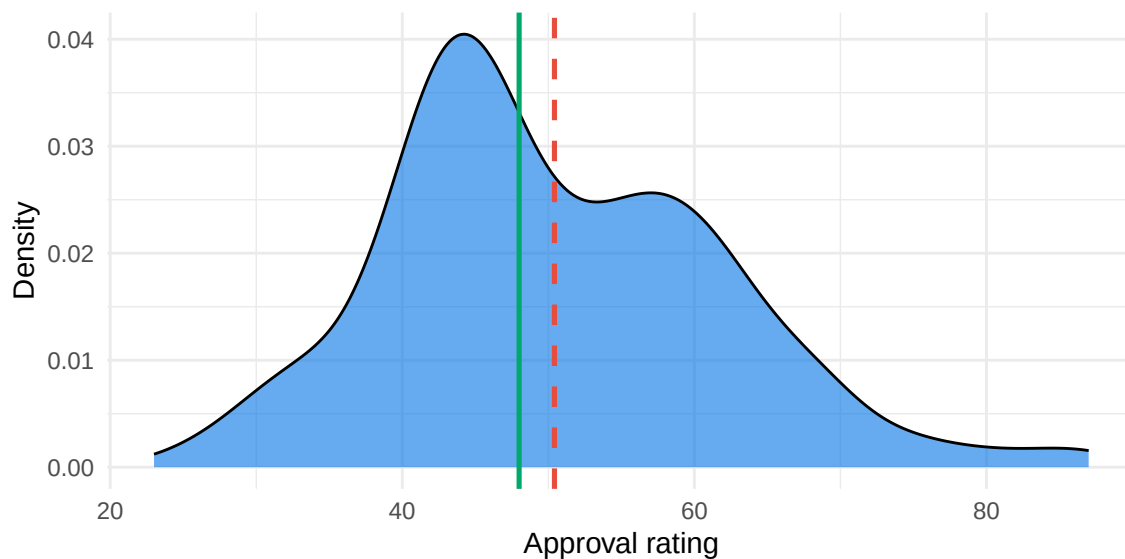


Figure 4: Figure 2.4: Density plot of presidential approval ratings, with the mean (red dashed) and median (green) marked.

What this code does: `geom_density()` draws a smoothed estimate of the distribution. `geom_vline()` adds vertical reference lines at chosen x-values: here, the mean and median. Annotating summary statistics directly onto a distribution is one of the clearest ways to convey what is "typical" and how the choice of measure changes the answer.

3 Using AI for This Material

Where AI Helps This Week

- Composing a dplyr pipeline from a verbal description, especially long chains of `filter()` into `group_by()` into `summarise()`.
- Explaining the differences between `filter()`, `select()`, and `slice()` and when each is the right verb.

- Sketching what a `group_by() %>% summarise()` chain will return before you run it.

Where AI Gets This Wrong This Week

- Choosing the mean versus the median for a specific variable requires looking at the distribution, which AI cannot do.
- Deciding whether to drop NAs or impute them depends on why the data are missing in your particular dataset, which AI cannot diagnose.
- It will sometimes assert the 68-95-99.7 rule applies when your variable is skewed, bounded, or discrete and the rule does not actually hold.

A prompt that works for this week:

Write a dplyr pipeline that takes a data frame called `polls`, filters to 2024 polls in swing states, groups by `state`, and returns the mean and median `dem_share` per state, ordered by mean descending.

A prompt that misleads:

Should I report the mean or the median for income in my dataset?

Why it misleads: choosing between mean and median requires inspecting the distribution's skew and outliers, which AI cannot do without seeing your data — it will default to a generic “use median for skewed data” without knowing whether yours is.

Where AI Helps This Week

- Writing long `case_when()` ladders with many branches without dropping a condition or mismatching types.
- Suggesting natural cutpoints (quartiles, conventional age brackets, income deciles) when converting a continuous variable to categories.
- Explaining what `if_else()` returns when the two branches have different types and why R complains.

Where AI Gets This Wrong This Week

- Choosing between an experimental and an observational design for a specific research question depends on what you can manipulate, what is ethical, and what data exist — judgments AI cannot make for you.
- It will not flag when a chain of `mutate()` calls is implicitly p-hacking by searching for

a specification that "works."

- It will turn a design question into a code request and produce a regression that looks causal even when the comparison is not valid.

A prompt that works for this week:

Write a `case_when()` that recodes age (a numeric column in years) into a factor called `age_group` with levels Under 30, 30 to 44, 45 to 64, and 65 and over.

A prompt that misleads:

I have data on districts that adopted voter ID laws and districts that did not. Write me the code to estimate the causal effect of voter ID on turnout.

Why it misleads: the request collapses a design question (are the groups comparable? is there a parallel-trends assumption?) into a code request, and AI will happily produce a regression that looks causal but is not.

4 Common Mistakes and How to Avoid Them

Mistake 1: Forgetting `na.rm = TRUE`

If any value in a vector is NA, then `mean(x)`, `median(x)`, and `sd(x)` all return NA. This is R protecting you, not failing. Add `na.rm = TRUE` when you intend to ignore missingness, but only after you have checked how much data is actually missing.

Mistake 2: Reporting the Mean for Skewed Data

When campaign contributions are reported as means, the picture is misleading. A handful of large donors pulls the mean far above what the typical donor gives. Always visualize the distribution first. If the histogram has a long tail, report the median, or report both and explain why they differ.

Mistake 3: Treating Correlation as Causation

This cannot be emphasized enough. Two variables can move together because one causes the other, because the other causes the one, because both are caused by something else, or by pure chance. Correlation alone cannot distinguish among these. Careful research design — the subject of Week 4 — is what lets us make causal claims.

Mistake 4: Using == to Check for Missing Values

The expression `x == NA` does not return `TRUE` when `x` is missing; it returns `NA`. Use `is.na(x)` to identify missing values and `!is.na(x)` to identify present ones.

Mistake 1: Using summarise() When You Meant mutate()

`summarise()` collapses a dataset to one row per group. `mutate()` adds a column without changing the number of rows. Students often write `summarise(age_group = case_when(...))` and watch every other column disappear. If you want each respondent to receive a new value, the verb is almost always `mutate()`.

Mistake 2: Forgetting the TRUE Catch-All in case_when()

If none of your conditions in `case_when()` match a row, that row gets `NA`. Always end with `TRUE ~ "Other"` or a similar safety net, unless you genuinely want missing values for unmatched cases. This is the single most common silent bug in tidyverse code.

Mistake 3: Treating Observational Patterns as Causal

A correlation between two variables in an observational dataset is consistent with many causal stories, including the story that some third factor produced both. No amount of variable creation can fix this. `mutate()` cannot rescue an observational comparison from confounding. Only a credible research design can.

Mistake 4: Overinterpreting Natural Experiments

Natural experiments live or die by the as-if-random assumption. Before trusting one, ask: could the units have anticipated the assignment? Could they have manipulated it? Are there other channels through which the assignment could affect the outcome? A lottery is only as clean as its implementation, and many designs that look quasi-random under one assumption look quite different under another.

5 R Functions Reference

Function	Purpose	Example
<code>names(df)</code>	List column names	<code>names(states)</code>
<code>filter()</code>	Keep rows matching conditions	<code>filter(party == "D")</code>
<code>select()</code>	Keep specified columns	<code>select(name, vote)</code>
<code>arrange()</code>	Sort rows	<code>arrange(desc(votes))</code>
<code>is.na(x)</code>	Test for missing values	<code>is.na(income)</code>
<code>!is.na(x)</code>	Test for non-missing values	<code>filter(!is.na(income))</code>

Table 1: Working with Real Data (Segment A)

Function	Purpose	Example
<code>mean(x)</code>	Arithmetic mean	<code>mean(x, na.rm = TRUE)</code>
<code>median(x)</code>	Median	<code>median(income)</code>
<code>sd(x)</code>	Standard deviation	<code>sd(approval)</code>
<code>var(x)</code>	Variance	<code>var(scores)</code>
<code>n()</code>	Count rows within <code>summarise()</code>	<code>summarise(n = n())</code>
<code>group_by()</code>	Group data for calculations	<code>group_by(state)</code>
<code>summarise()</code>	Compute summary statistics	<code>summarise(m = mean(x))</code>
<code>ggplot()</code>	Initialize a plot	<code>ggplot(df, aes(x = v))</code>
<code>geom_histogram()</code>	Add a histogram layer	<code>geom_histogram(bins = 30)</code>
<code>geom_density()</code>	Add a density layer	<code>geom_density()</code>
<code>geom_vline()</code>	Add a vertical reference line	<code>geom_vline(xintercept = 50)</code>

Table 2: Summary Statistics and Visualization (Segment B)

Function	Purpose	Example
<code>mutate()</code>	Create or modify columns	<code>mutate(margin = dem - rep)</code>
<code>if_else()</code>	Two-way recoding	<code>if_else(age >= 65, "Senior", "Not")</code>
<code>case_when()</code>	Multi-way recoding	<code>case_when(age < 30 ~ "Young", ...)</code>
<code>pull()</code>	Extract a column as a vector	<code>pull(med)</code>
<code>n_distinct()</code>	Count unique values	<code>n_distinct(state)</code>
<code>across()</code>	Apply over multiple columns	<code>across(ends_with("vote"), ...)</code>
<code>recode()</code>	Swap specific values	<code>recode(x, "old" = "new")</code>
<code>count()</code>	Tally rows per group	<code>count(treatment, voted)</code>

Table 3: Key R Functions for This Class

6 Practice Problems

The following questions are designed to test your understanding of the material in this chapter. Work through them before class, and bring your questions to discussion.

Question 1

You load a dataset of 1,000 survey respondents and find that 220 of them have a missing value for the `income` variable. You decide to compute the mean income with `mean(income, na.rm = TRUE)`. What assumption are you making about the 220 missing cases? Under what circumstances would this assumption be problematic?

Question 2

Write a single tidyverse pipeline that takes the `states` data frame, keeps only Southern states from the 2020 election, sorts from highest to lowest Democratic vote share, and shows the first five rows of `state`, `region`, and `democratic_vote_share`. Label each step with a comment explaining what it does.

Question 3

A news article reports that "the average American household has \$1.06 million in net worth." A friend who reads this concludes that most Americans are doing well financially. Explain in two or three sentences why this conclusion is probably wrong, using the language of mean, median, and skewness.

Question 4

Researchers report that across U.S. states, the correlation between the number of Starbucks locations and average SAT scores is $r = 0.41$. Propose at least two non-causal explanations for this correlation. What would you need to do to test whether Starbucks locations actually cause SAT improvements?

Question 5 (Computational)

Using the `states` data, write a grouped summary that, for each combination of `region` and `year`, reports the number of states, the mean Republican vote share, and the standard deviation of the Republican vote share. Which region-year combinations show the most internal disagreement (largest SD), and which the most uniformity (smallest SD)? Use `arrange()` to find out.

For Further Study

This chapter has introduced the working pattern of modern political data analysis: load a real dataset, wrangle it with tidyverse verbs, handle missingness honestly, and summarize it with statistics that match the shape of the distribution. Students who want to go deeper might explore Hadley Wickham and Garrett Grolemund's *R for Data Science*, which is freely available online and covers tidyverse in depth. For a careful treatment of why correlation is not causation, Judea Pearl and Dana Mackenzie's *The Book of Why* is accessible and engaging. In the coming classes we will build on this foundation with summary statistics and by creating new variables with `mutate()`, and we will turn our attention to the question of when data can support causal claims.

Question 1

You have a dataset of congressional districts with columns `dem_votes`, `rep_votes`, and `incumbent_party`. Using `mutate()` and `case_when()`, create a variable `competitiveness` that classifies each district as "Safe Dem," "Lean Dem," "Toss-Up," "Lean Rep," or "Safe Rep" based on the Democratic vote share. Explain the cutpoints you chose and what theoretical assumptions they encode.

Question 2

A campaign claims that voters who received its mailers turned out at a higher rate than voters who did not. Why is this comparison unconvincing as evidence that the mailers worked? Describe two confounders that could produce the same pattern. What design would the campaign need to use to make a credible causal claim?

Question 3

Compare a cross-sectional survey of American voters in 2024 to a panel survey that interviewed the same voters in 2020, 2022, and 2024. What question can the panel answer that the cross-section cannot? What practical disadvantages does the panel design carry?

Question 4 (Computational)

Create a simulated dataset of 500 voters with `age`, `party`, and a randomly assigned `treatment` indicator. Use `mutate()` and `case_when()` to create an `age_group` variable with four categories. Use `count()` to verify that the treatment and control groups have similar age-group distributions. What would you conclude if they did not?

For Further Study

Wickham and Grolemund's *R for Data Science* provides an extended treatment of the tidyverse transformation verbs introduced here, with many more worked examples. For research design, Angrist and Pischke's *Mostly Harmless Econometrics* remains the most accessible

introduction to modern causal inference, while Dunning's *Natural Experiments in the Social Sciences* catalogs the most influential as-if-random designs in political science. As you read these works, notice how often the analytical move that makes a study compelling is a small, careful transformation of a variable, combined with a research design that gives that transformation meaning.